EE1021 Term Project Guidelines

Fall 2026-2027 · Programming for Engineers

Basri Erdoğan

1 What You Will Build

In a group of **3 students**, you will write a Python program that solves a practical problem. The program must use **object-oriented programming** (classes and inheritance) and **file input/output** (it saves its data and loads it again). The project is **20%** of your course grade.

2 Key Dates

Step	What you do	Date
Topics announced	Read these guidelines; form your group	**14 October 2026**
Group registration	E-mail your 3 names, student numbers and topic to Res. Asst. Ahmet Aytuğ Ayrancı (ahmet.ayranci@iku.edu.tr)	**26 October 2026, 23:59**
Submission	Upload source code and report to CATS (one upload per group)	**29 November 2026, 23:59**
Presentation	Live demonstration and Q&A in your own lab section	Week 12 lab, **3 December 2026**

Late submissions are not accepted, and there is no make-up for the project.

3 When Each Topic Is Taught

You can start before every topic is taught. Add each part when you learn it:

Requirement	Lecture	Date
`try` and `except` (no crashes)	L06 Errors and exceptions	14 October
Classes, objects and methods	L07 Object-oriented programming I	21 October
Inheritance	L08 Object-oriented programming II	11 November
NumPy and matplotlib (optional)	L09 Data and visualisation	18 November
Reading and writing files	L10 File input/output and CSV	25 November

4 Rules for Groups and Topics

- Each group has exactly **3 members**. If the class size makes this impossible, the course assistant will form the remaining groups.
- Choose a topic from the list in Section 10 or propose your own. Your own idea must be approved by the course assistant.
- Each topic can be taken by **one group only**. The first group to register a topic gets it. If your topic is taken, choose another one.
- After registration you cannot change your group or topic without the instructor's approval.

5 Technical Requirements

Your program **must** meet all of these requirements:

1. **Language:** Python 3. The program starts with one command, for example `python main.py`.
2. **Classes:** at least **3 classes that you write yourself**, each with its own attributes and methods.
3. **Inheritance:** at least **one base class with at least one subclass** that adds or changes behaviour (for example `Component` → `Resistor`, `Capacitor`).
4. **Files:** the program **saves its data to a file** (`.txt` or `.csv`) and **loads it again** when it starts. Example: the inventory still shows yesterday's stock after you close and reopen the program.
5. **No crashes:** wrong input (for example `abc` where a number is expected, or a missing file) must give a clear message, not a traceback. Use `try` and `except`.
6. **Menu:** a text menu in the terminal that lets the user choose each feature.

Optional: you may use extra packages such as `matplotlib` for plots or `tkinter` for windows. A graphical interface does not give extra points; the requirements above come first.

6 What You Submit

Upload **one ZIP file per group** to CATS. Name it `EE1021_Project_GroupNN.zip` (NN = your group number). It contains:

1. **Source code:** all `.py` files and the data files your program needs. Use relative file paths (for example `data/inventory.csv`), not paths from your own computer.
2. **`README.txt`:** group members, how to start the program, and the Python packages it needs.
3. **Report (PDF, 4-6 pages)** with these sections:
    1. Problem: what your program does and who uses it.
    2. Class design: each class, its attributes and methods, and the inheritance relation (a simple class diagram is enough).
    3. File format: what you save and how one line of the file looks.
    4. Error handling: which wrong inputs you handle and how.
    5. User manual: screenshots of the main menu and one example run.
    6. Work distribution: which member wrote which part.
    7. AI use: which AI tools you used and for what (see Section 8). Write "none" if you used none.

7 Presentation and Q&A

- Each group has about **10 minutes**: 5 minutes of live demonstration and 5 minutes of questions.
- Run the program **from your submitted code**. Show adding data, saving, closing, restarting and loading.
- **Every member** must be able to explain any part of the group's code. The instructor will choose lines for each member to explain and will ask each member for a small live change (for example "add a menu option" or "change this message").
- Your Q&A result decides how many of the group's points you receive (see Section 9). A member who cannot explain the group's code receives **zero** for the project, even if the program works.

8 Use of AI Tools

These rules come from the course syllabus.

- **Allowed:** asking an AI tool to explain a concept, a Python feature or an error message; getting feedback on the readability of code you wrote yourself; language help for the report.
- **Not allowed:** generating your project's code or solution with AI and submitting it as your own work.
- **Disclosure:** state in the report how you used AI tools.
- **Check:** in the Q&A you explain and change your own code live. Code you cannot explain earns no credit.

9 Grading

Part	Points	Graded per
Program: requirements met, works without crashing, clear classes and names	50	group
Report: design, file format, user manual, work distribution, AI statement	20	group
Presentation and Q&A: demonstration, explaining and changing the code	30	member

Each member receives the group's points only as far as that member can explain and change the code:

Your project grade = (Program + Report) × Q&A multiplier + your Q&A points

Your result in the Q&A	Q&A multiplier
You explain the code **and** make the live change correctly	× 1
You explain the code, but you need help with the live change	× 0.5
You cannot explain the code	× 0 (project grade = 0)

Example. The group's program gets 45 and the report 18, so the group has 63 points.

Member	Q&A result	Calculation	Project grade
A	explains and changes the code; Q&A 27	63 × 1 + 27	90
B	explains, needs help with the change; Q&A 15	63 × 0.5 + 15	46.5
C	cannot explain the code	0	0

10 Project Ideas

The hints show one possible class design. You may design your classes differently.

10.1 Resistor Color Code Calculator

- Enter 4 or 5 color bands and get the resistance and tolerance; enter a resistance and get the color bands.
- Save the calculation history to a file and load it at start.
- *Classes:* `Resistor` (base), `FourBandResistor`, `FiveBandResistor`, `CalculatorApp`.

10.2 Student Grade Management System

- Add and remove students, enter midterm and final grades, calculate letter grades and averages.
- Save and load the student data as a CSV file.
- *Classes:* `Person` (base), `Student`, `Course`.

10.3 Complex Number Calculator

- Add, subtract, multiply and divide complex numbers; convert between rectangular and polar form.
- Save the results of a session to a file and reload them.
- *Classes:* `Number` (base), `ComplexNumber`, `Calculator`.

10.4 Electronic Component Inventory

- Add resistors, capacitors and ICs; update stock; search by type or value; warn when the stock is low.
- Save and load the inventory as a CSV file.
- *Classes:* `Component` (base), `Resistor`, `Capacitor`, `Inventory`.

10.5 Simple Circuit Solver (Series and Parallel)

- Build a circuit from series and parallel resistors; calculate the equivalent resistance and the current for a given voltage.
- Save a circuit to a file and load it again.
- *Classes:* `CircuitElement` (base), `Resistor`, `VoltageSource`, `Circuit`.

10.6 Bank Account Manager

- Create savings and checking accounts; deposit, withdraw and transfer; show the transaction history.
- Save accounts and transactions to files.
- *Classes:* `Account` (base), `SavingsAccount`, `CheckingAccount`, `Bank`.

10.7 Quiz Generator

- Read a question bank from a file; pick N random questions; admin mode adds questions; student mode takes the quiz and shows the score.
- *Classes:* `Question` (base), `MultipleChoiceQuestion`, `TrueFalseQuestion`, `QuizManager`.

10.8 Matrix Operations Library

- Add, subtract and multiply matrices; transpose; determinant of 2×2 and 3×3 matrices.
- Read matrices from a file and save results.
- *Classes:* `Matrix` (base), `SquareMatrix` (adds the determinant), `MatrixApp`.

10.9 Personal Budget Tracker

- Add income and expenses with categories; show the balance and a summary per category.
- Save and load all entries as a CSV file.
- *Classes:* `Transaction` (base), `Income`, `Expense`, `BudgetManager`.

10.10 Signal Generator

- Generate sine, square and triangle waves from a frequency and an amplitude.
- Save the samples to a CSV file; plot them with `matplotlib` or as text with `*` characters.
- *Classes:* `Signal` (base), `SineWave`, `SquareWave`, `TriangleWave`, `SignalPlotter`.

10.11 Library Management System

- Add and remove books; register members; lend and return books with due dates.
- Save books, members and loans to files.
- *Classes:* `LibraryItem` (base), `Book`, `Member`, `LibrarySystem`.

10.12 Contact Book

- Store name, phone, e-mail and address; search, edit and delete contacts.
- Save and load the contacts as a file.
- *Classes:* `Contact` (base), `PersonalContact`, `BusinessContact`, `AddressBook`.

10.13 Tic-Tac-Toe or Connect 4 with Saved Games

- Two-player mode; detect a win or a draw.
- Save the board to a file and continue the game later.
- *Classes:* `Player` (base), `HumanPlayer`, `ComputerPlayer`, `Game`.

10.14 Weather Data Analyzer

- Read daily temperature and rainfall from a CSV file; calculate monthly averages; find the hottest and coldest days; write a summary report to a file.
- *Classes:* `AnalysisTool` (base), `TemperatureAnalyzer`, `RainfallAnalyzer`, `WeatherSystem`.